Homework 1 Report

Josue Rocha - jrocha3@nd.edu

Design Decisions

In order to extract information from NVD, I utilized the JSON data feeds. In particular, I worked off of the script *parser.py* provided in class. Since this script was already printing its results, I was able to easily determine what the next steps should be. I immediately worked on taking the output from the *parser.py* code, and figuring out how to extract the relevant information and store it in an SQLite database. To do this, I utilized the fact that the outputs from *parser.py* were a series of dictionaries. This made it easy to split up the outputs and store only the information I was interested in. I also implemented checks for edge cases such as outputs that didn't specify any affected versions. For parsing the pom.xml files, I utilized the xml.etree module to do a lot of the heavy lifting. This did provide one of the biggest technical challenges I encountered since I did not have any experience with the xml.etree module prior. As such, I had to do a lot of digging, and trial and error in order to implement it. To match CVE information to dependencies in a pom file, I utilized the Levenshtein module to calculate the Levenshtein distance between the groupId collected from the pom.xml file and the cpe stored in the knowledge base. I chose to not utilize the description given by the JSON data feeds in this comparison because they seemed to be rather unreliable. Given the fact that the description varied greatly in size and were written in human language, it would very often lead to unreliable Levenshtein distances. After a very brief trial and error period, I settled on five as my heuristic for the Levenshtein distance since it would yield consistent results with the data I was working with.

Knowledge Base Statistics

The submission revolves around persisting data as a way to efficiently manage large datasets. Instead of repeatedly requesting and parsing massive JSON data feeds from NVD, I store only the relevant information (CVE IDs, CPEs, etc.) in a structured SQLite database. By utilizing persistent data, the program is able to quickly query the knowledge base whenever a new pom.xml file needs to be analyzed. This not only reduces computational overhead but also ensures that the results are consistent, reproducible, and easily extensible for future improvements.

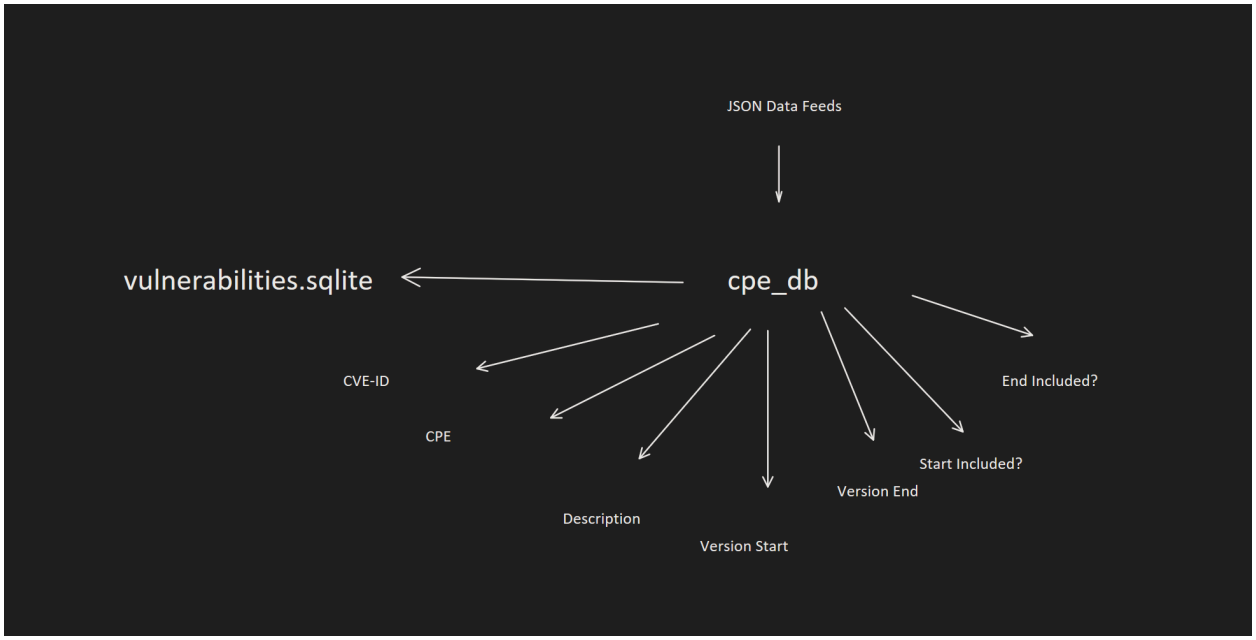

Figure 1:

As you can see in Figure 1, the program utilizes the JSON Data Feeds to build the vulnerabilities.sqlite database. The database consisted of only one table title cpe_db. This table has size rows, two of which store information to be used for matching dependencies and vulnerabilities. Another row is used to store the CVE-ID, and the final four rows are used to store information regarding the versions affected by a vulnerability. Given the inconsistency with information regarding affected versions in the JSON Data Feeds, I added those extra rows so that I would be able to later piece together which versions were affected when it came time to provide an output. The last time the knowledge base was updated was at approximately 9:30pm on 09/09/2025. Due to time constraints however, the process had to be cut short. At the time of termination, a total of 1,496,381 entries were able to be processed.